



Encapsulamiento

El **encapsulamiento** es un mecanismo que permite la definición de módulos de software que pueden ser utilizados como "cajas negras", esto es, sabiendo **qué** hacen sin saber **cómo** lo hacen.

Cada módulo de software tiene una **interfaz** que le permite comunicarse con los demás módulos del sistema.

Un sistema complejo se construye a partir de módulos que se comunican a través de su interfaz.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1



Encapsulamiento en la POO

En la POO cada **clase** es un módulo de software cuya implementación debe quedar escondida, de modo que pueda ser modificada sin afectar al resto de las clases.

La **interfaz** está constituida por la **signatura de los servicios**.

La construcción de un sistema orientado a objetos va a estar conformado por una colección de clases relacionadas entre sí.

Algunas de las clases son **proveedoras** de servicios usados por otras clases **clientes**.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1



Encapsulamiento en la POO

Entre cada clase proveedora y sus clientes se establece un **contrato** que define la **funcionalidad** y las **responsabilidades**.

El código de cada clase **proveedora** se escribe a partir del **contrato** especificado por los requerimientos y el diseño, sin conocer las clases que la usan.

El programador de la clase **cliente** usa a la clase **proveedora**, conoce su **interfaz** y el **contrato**, pero no la implementación.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1



Encapsulamiento en Java

En Java el programador de una clase establece el **nivel de encapsulamiento** usando los **modificadores de acceso**.

Los constructores, comandos y consultas especificados por el diseñador en el diagrama, se declaran como **públicos** y son visibles para las clases clientes.

Los atributos se declaran como **privados** y quedan encapsulados en la **clase**, de modo que puedan ser modificados sin provocar un impacto en el resto de las clases que conforman el sistema.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1



Arreglos

Un **arreglo** es una **estructura de datos** homogénea y lineal.

Es una **estructura de datos** porque agrupa a varios valores, que pueden ser accedidos individualmente.

Es **homogénea** porque todos los valores son del mismo tipo.

Es **lineal** porque todos los valores ocupan posiciones continuas, de modo que cada uno tiene uno que le sigue, excepto el último y todos tienen uno anterior, excepto el primero.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1



Arreglos

Un lenguaje de programación que soporte arreglos debe brindar operaciones para:

- Declarar una variable
- Crear la estructura
- Acceder a sus componentes

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Arreglos en Java

Un arreglo queda asociado a una única variable.

Cada valor es un **elemento** o **componente** del arreglo y puede accederse a través de un **subíndice**.

El subíndice representa la posición relativa del elemento dentro del arreglo.

En Java un subíndice toma valores entre 0 y la cantidad de componentes del arreglo menos 1.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Estación meteorológica

En una estación meteorológica se miden y registran valores de distintas variables meteorológicas, que luego se usan para hacer predicciones, entre ellas las temperaturas mínimas de un período.

A partir de este registro es necesario:

- Calcular la mayor temperatura mínima registrada
- Calcular el promedio de las temperaturas
- Calcular en cuántos días heló
- Decidir si hubo heladas



Estación meteorológica

8.0	3.5	12.0	-1.0	3.5	10.0	-2.0
-----	-----	------	------	-----	------	------

Mayor temperatura mínima = 12,0
Promedio = 4,8
Cantidad de heladas = 2
Hubo Heladas = true

En una estación meteorológica se miden y registran valores de distintas variables meteorológicas, que luego se usan para hacer predicciones, entre ellas las temperaturas mínimas de un período.

A partir de este registro es necesario:

- Calcular la mayor temperatura mínima registrada
- Calcular el promedio de las temperaturas
- Calcular en cuántos días heló
- Decidir si hubo heladas



Estación meteorológica

La clase **TempMinEstacion** encapsula la representación de la estructura de datos que mantiene las temperaturas mínimas registradas en una estación meteorológica en un período de n días.

El diseñador decidió que los datos se mantengan en un **arreglo de n componentes**, cada una de las cuales corresponde a la temperatura mínima de un día.

Como la clase encapsula a la estructura de datos, el arreglo no es visible fuera de la clase **TempMinEstacion**.

Las clases Clientes pueden crear objetos de clase **TempMinEstacion** y enviarles mensajes sin conocer la representación de los datos ni la implementación de los servicios.

Desde las clases Clientes el primer día se referencia con el valor 1 y el último día del período corresponde al valor n.

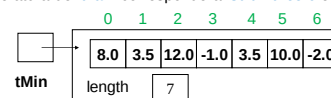
```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Estación meteorológica

TempMinEstacion
<<atributos de instancia>> float [] tMin
<<Constructores>> TempMinEstacion(cant: entero)
<<Comandos>> establecerTempMin(dia:entero, t:real)

TempMinEstacion(cant: entero) Crea una estructura para mantener las temperaturas de cant días, requiere cant>0
establecerTempMin (dia:entero,t:real) requiere 1<=dia<=cantDias()

La temperatura del **día 1** corresponde al **subíndice 0** en el arreglo.



```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Estación meteorológica

TempMinEstacion
 <<atributos de instancia>>
 float [] tMin

<<Consultas>>
 obtenerTempMin(dia: entero): real
 cantDias(): entero
 mayorTempMin () : real
 promedioTempMin () : real
 cantHeladas(): entero
 huboHeladas () : boolean

Todas las consultas que procesan la estructura requieren que se haya asignado una temperatura a cada día.
 El período tiene al menos un día y el primer día se referencia con 1 desde las clases cliente.

obtenerTempMin (dia : entero): real
 requiere 1<=dia<=cantDias()

mayorTempMin () : real
 Retorna la mayor temperatura del período

promedioTempMin () : real
 Retorna el promedio de las temperaturas del período

cantHeladas(): entero
 Cuenta la cantidad de días que heló

huboHeladas () : boolean
 Decide si hubo heladas

13

Estación meteorológica

```
class TempMinEstacion {
    /* Todas las consultas que procesan la estructura requieren
    que se haya asignado una temperatura a cada día. El primer día
    es el día 1.El periodo tiene al menos un día.*/
    private float [] tMin;

    //Constructor
    /*Crea una estructura para mantener las temperaturas de cant
    dias, cant>0 */
    public TempMinEstacion(int cant){
        tMin = new float[cant];
    }
    ...
}
```

14

Estación meteorológica

```
//Comandos
public void establecerTempMin(int dia, float t){
    /*Requiere 1<=dia<=cantDias()*/
    tMin[dia-1]=t;
}
```

La temperatura de **dia** corresponde al **subíndice dia - 1** en el arreglo.

15

Estación meteorológica

```
// Consultas
public float obtenerTempMin(int dia){
    /*Requiere 1<=dia<=cantDias()*/
    return tMin[dia-1];
}

public int cantDias(){
    return tMin.length;
}
```

16

Estación meteorológica

```
public int cantHeladas(){
    //Cuenta la cantidad de días que heló
    int cant = 0;
    for (int i = 0; i<cantDias(); i++)
        if (tMin[i] < 0)
            cant++;
    return cant;
}
```

17

Estación meteorológica

```
public boolean huboHeladas(){
    /*Retorna true si algún día heló */
    boolean encuentre = false;
    int i = 0;
    while ((i < cantDias()) && (!encontre))
        if (tMin[i] < 0)
            encuentre = true;
        else
            i++;
    return encuentre;
}
```

18

Estación meteorológica

El arreglo está **encapsulado** en la clase, todo el acceso desde la clase cliente se realiza a través de los servicios ofrecidos por la clase proveedora.

La cantidad de elementos del arreglo se define en el momento de la creación y corresponde a la cantidad de días del período.

Para las clases cliente, el primer día se denota con 1.

La entrada y salida se realiza desde la clase cliente.

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1

19

Estación meteorológica

```
class TestUnaSemana {
    public static void main(String[] args) {
        int cant = 7;
        int chSur, chNor;
        TempMinEstacion estSur, estNor, est;
        estSur = leerTempMinEst(cant);
        estNor = leerTempMinEst(cant);
        chSur = estSur.cantHeladas();
        chNor = estNor.cantHeladas();
        if (chSur >= chNor)
            est = estSur;
        else
            est = estNor;
        mostrarTempMinEst(est);
    }
}
```

20

Estación meteorológica

TempMinEstacion
<<atributos de instancia>> float [] tMin
<<Consultas>> contarMayores(t: real): entero primerMayor(t: real): real ultimaMayor(t: real): real diaPrimerMayor(t: real): entero
Todas las consultas que procesan la estructura requieren que se haya asignado una temperatura a cada día. El período tiene al menos un día y el primer día se referencia con 1 desde las clases cliente.

contarMayores(t: real): entero Computa la cantidad de días con temperaturas mayores a t
primerMayor(t: real): real Computa la primera temperatura mayor a t, si no hay una mayor a t retorna t
ultimoMayor(t: real): real Computa la última temperatura mayor a t, si no hay una mayor a t retorna t
diaPrimerMayor(t: real): entero Computa el día en el que se produjo la primera temperatura mayor a t. Si no hubo ninguna retorna 0

21

Estación meteorológica

- Computar la cantidad de días en los que la temperatura fue mayor a un valor dado.

8	3.5	11	-1	13.5	10	-2
---	-----	----	----	------	----	----

- si $t=5.5$ debe computar 4
- si $t=15$ debe computar 0
- si $t=-5$ debe computar 7

22

Estación meteorológica

- Computar la cantidad de días en los que la temperatura fue mayor a un valor dado.

8	3.5	11	-1	13.5	10	-2
t	5.5					
contador	0					

23

Estación meteorológica

- Computar la cantidad de días en los que la temperatura fue mayor a un valor dado.

8	3.5	11	-1	13.5	10	-2
t	5.5					
contador	1					

24

Estación meteorológica

Estación meteorológica

Estación meteorológica

Estación meteorológica

Estación meteorológica

Estación meteorológica

```

Algoritmo primerMayor
DE t
DS pri
para cada día del periodo y mientras no encuentre una
temperatura mayor a t
    si la temperatura del día es mayor a t
        pri = temperatura del día
    
```

8	3.5	11	-1	13.5	10	-2
---	-----	----	----	------	----	----

primerMayor(15) computa 15 y termina

37

Estación meteorológica

```

public float primerMayor(float t){
    /*Computa la primera temperatura mayor a t, si no hay una
    mayor a t, retorna t*/
    float pri = t;
    int i = 0; boolean encontro = false;
    while (i < cantDias() && !encontro){
        encontro = tMin[i] > t;
        i++;
    }
    if (encontro)
        pri = tMin[i-1];
    return pri;
}

```

8	3.5	11	-1	13.5	10	-2
---	-----	----	----	------	----	----

primerMayor(9) computa 11 y termina
primerMayor(15) computa 15 y termina

38

Estación meteorológica

```

public float primerMayor(float t){
    /*Computa la primera temperatura mayor a t, si no hay una
    mayor a t, retorna t*/
    float pri = t;
    int i = 0;
    while (i < cantDias() && pri == t)
        if (tMin[i] > t)
            pri = tMin[i];
        else
            i++;
    return pri;
}

```

39

Estación meteorológica

Computa la última temperatura mayor a t, si no hay ninguna retorna t.

8	3.5	11	-1	13.5	10	-2
---	-----	----	----	------	----	----

ultimaMayor(9)

Computa 10

40

Estación meteorológica

```

public float ultimaMayor(float t){
    //Computa la última temperatura mayor a t o t
    float ult = t;
    int i = cantDias() - 1;
    while (i >= 0 && ult == t){
        if (tMin[i] > t)
            ult = tMin[i];
        else
            i--;
    }
    return ult;
}

```

41

Estación meteorológica

- Computar el número de día que corresponde la primera temperatura mayor a una dada. Si en ningún día del período la temperatura es mayor a la dada, retorna 0.

8	3.5	11	-1	13.5	10	-2
---	-----	----	----	------	----	----

diaMayor(9)

42

Estación meteorológica

TempMinEstacion
<<atributos de instancia>> float [] tMin
<<Consultas>> equals(e: TempMinEstacion): boolean
Todas las consultas que procesan la estructura requieren que se haya asignado una temperatura a cada día. El período tiene al menos un día y el primer día se referencia con 1 desde las clases cliente.

equals(e: TempMinEstacion): boolean
Computa true si y solo si coinciden las temperaturas de las dos estaciones día a día

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

Estación meteorológica

```
public boolean equals(TempMinEstacion e){
    /*Computa true si y solo si coinciden las temperaturas de las
    dos estaciones día a día*/
    boolean iguales = e != null && cantDias() == e.cantDias();
    float t; int i = 0;
    while (i < cantDias() && iguales){
        t = e.obtenerTempMin(i+1);
        iguales = t == tMin[i];
        i++;
    }
    return iguales;
}
```

Tenemos que tener en cuenta el mapeo establecido para la clase cliente.

50

Estación meteorológica

```
public boolean equals(TempMinEstacion e){
    /*Computa true si y solo si coinciden las temperaturas de las
    dos estaciones día a día*/
    boolean iguales = e != null && cantDias() == e.cantDias();
    float t; int i = 0;
    while (i < cantDias() && iguales){
        t = e.obtenerTempMin(i+1);
        iguales = t == tMin[i];
        i++;
    }
    return iguales;
}
```

El parámetro **e** mantiene una referencia a un objeto de clase **TempMinEstacion**, puede recibir cualquiera de los mensajes provistos por la clase.

51

Estación meteorológica

```
public boolean equals(TempMinEstacion e){
    /*Computa true si y solo si coinciden las temperaturas de las
    dos estaciones día a día*/
    boolean iguales = e != null && cantDias() == e.cantDias();
    float t; int i = 0;
    while (i < cantDias() && iguales){
        t = e.obtenerTempMin(i+1);
        iguales = t == tMin[i];
        i++;
    }
    return iguales;
}
```

La variable **tMin** mantiene una referencia a un arreglo

52

Algunos Casos de Prueba

6.2	2.1	4.2		5.5	11.1	-4.2	9.0	5.5	11.1	-4.2	9.0
6.2	2.1	4.2	9.0	5.5	0.1	-4.2	3.0	5.5	11.1	-4.2	9.0

```
01100
10011
10110
01110
01100
10011
10110
1001
1 11
0 0
1
```

53

Estación meteorológica

TempMinEstacion	dosSeguidos(t: real): boolean Decide si hubo al menos dos días seguidos con temperaturas mayores a t
<<atributos de instancia>> float [] tMin	huboAlMenosNMayores(t:real,n:entero):boolean Decide si hubo al menos n días con temperaturas mayores a t.
<<Consultas>> dosSeguidos(t: real): boolean huboAlMenosNMayores(t:real,n:entero):boolean huboNMayores(t:real,n:entero):boolean huboNConMayores(t:real,n:entero, cant: entero): boolean	huboNMayores(t:real,n:entero):boolean Decide si hubo exactamente n días con temperaturas mayores a t.
Todas las consultas que procesan la estructura requieren que se haya asignado una temperatura a cada día. El período tiene al menos un día y el primer día se referencia con 1 desde las clases cliente.	huboNConMayores(t:real,n:entero):boolean Decide si hubo al menos n días consecutivos con temperaturas mayores a t.

54

Estación meteorológica

- Decide si la temperatura de dos días seguidos fue mayor a t.

7.2	2.2	8.1	-4.2	9.0	5.5	0.1	6.2	7.0
-----	-----	-----	------	-----	-----	-----	-----	-----

si t=6.0 debe computar true

si t=7.0 debe computar false

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

55

Estación meteorológica

Algoritmo dosSeguidos
 DE t
 para todos los días del periodo
 excepto el último
 y mientras no se verifica la propiedad
 si la temperatura del día y la del día siguiente son
 mayores a t
 se verifica la propiedad

Al implementar esta solución en Java es importante considerar la condición de terminación de la iteración, ya que el último elemento de la estructura no tiene un "siguiente".

```
01100
11
11
10
10
00
00
11
10
10
10
11
1
```

56

Estación meteorológica

```
public boolean dosSeguidos (float t){
    /*Decide si hay dos días seguidos con temperaturas mayores a t*/
    boolean dos = false;
    if (cantDias() >= 2)
        for (int i = 0; i < cantDias() - 1 && !dos; i++)
            if ((tMin[i] > t) && (tMin[i+1] > t))
                dos = true;
    return dos;
}
```

Recorrido NO Exhaustivo

```
00
11
10
10
00
11
10
10
1
0
1
0
1
```

57

Estación meteorológica

- Decidir si hubo al menos n temperaturas mayores a t.

7.2	6.2	2.1	5.5	-2.0
-----	-----	-----	-----	------

t=5.5 n=3

Se recorre toda la estructura y no se encuentran 3 valores mayores a 5.5.

7.2	2.2	8.1	-4.2	9.0	5.5	0.1	6.2	7.0
-----	-----	-----	------	-----	-----	-----	-----	-----

t=5.5 n=3

Se recorren los cinco primeros elementos, en los cinco primeros días hay 3 temperaturas mayores a 5.5.

4.0	3.0	9.0	0.1	6.2	7.0
-----	-----	-----	-----	-----	-----

t=5.5 n=3

Se recorre toda la estructura, recién al analizar el último día se puede decidir que hay 3 temperaturas mayores a 5.5.

```
01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
```

58

Estación meteorológica

Algoritmo huboAlMenosNMayores
 DE t, n
 para cada día
 y mientras no se encuentren n mayores
 si la temperatura es mayor a t
 incrementar un contador

```
01100
11
11
10
10
01100
10011
10110
01110
1001
1 11
0 0
1
```

59

Estación meteorológica

```
public boolean huboAlMenosNMayores(float t,int n){
    /*Decide si hubo al menos n temperaturas mayores a t*/
    int cont = 0;
    if (cantDias() >= n)
        for (int i=0; i < cantDias() && cont < n; i++)
            if (tMin[i] > t)
                cont++;
    return cont == n;
}
```

Recorrido NO Exhaustivo

```
00
11
11
10
10
00
00
11
10
10
10
1 11
0 0
1
```

60

Estación meteorológica

- Decide si hubo al menos n días consecutivos con temperaturas mayores a t .

↓

2.5	11.1	8.2	4.0	0.1	-0.2	4.8	5.2	5.1	-0.2
-----	------	-----	-----	-----	------	-----	-----	-----	------

si $t = 4.5$ y $n = 3$

```

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
    
```

67

Estación meteorológica

- Decide si hubo al menos n días consecutivos con temperaturas mayores a t .

↓

2.5	11.1	8.2	4.0	0.1	-0.2	4.8	5.2	5.1	-0.2
-----	------	-----	-----	-----	------	-----	-----	-----	------

si $t = 4.5$ y $n = 3$
cont ← 1

```

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
    
```

68

Estación meteorológica

- Decide si hubo al menos n días consecutivos con temperaturas mayores a t .

↓

2.5	11.1	8.2	4.0	0.1	-0.2	4.8	5.2	5.1	-0.2
-----	------	-----	-----	-----	------	-----	-----	-----	------

si $t = 4.5$ y $n = 3$
cont ← 2

```

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
    
```

69

Estación meteorológica

- Decide si hubo al menos n días consecutivos con temperaturas mayores a t .

↓

2.5	11.1	8.2	4.0	0.1	-0.2	4.8	5.2	5.1	-0.2
-----	------	-----	-----	-----	------	-----	-----	-----	------

si $t = 4.5$ y $n = 3$
cont ← 0

```

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
    
```

70

Estación meteorológica

- Decide si hubo al menos n días consecutivos con temperaturas mayores a t .

↓

2.5	11.1	8.2	4.0	0.1	-0.2	4.8	5.2	5.1	-0.2
-----	------	-----	-----	-----	------	-----	-----	-----	------

si $t = 4.5$ y $n = 3$
cont ← 0

```

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
    
```

71

Estación meteorológica

- Decide si hubo al menos n días consecutivos con temperaturas mayores a t .

↓

2.5	11.1	8.2	4.0	0.1	-0.2	4.8	5.2	5.1	-0.2
-----	------	-----	-----	-----	------	-----	-----	-----	------

si $t = 4.5$ y $n = 3$
cont ← 0

```

01100
10011
10110
01110
01100
10011
10110
01110
1001
1 11
0 0
1
    
```

72

Estación meteorológica

- Decide si hubo al menos n días consecutivos con temperaturas mayores a t .

2.5	11.1	8.2	4.0	0.1	-0.2	4.8	5.2	5.1	-0.2
-----	------	-----	-----	-----	------	-----	-----	-----	------

si $t = 4.5$ y $n = 3$

cont $\leftarrow 1$

```

00 00
01 00
02 01
03 01
04 10
05 10
06 10
07 10
08 10
09 10
10 10
11 10
12 10
13 10
14 10
15 10
16 10
17 10
18 10
19 10
20 10
21 10
22 10
23 10
24 10
25 10
26 10
27 10
28 10
29 10
30 10
31 10
32 10
33 10
34 10
35 10
36 10
37 10
38 10
39 10
40 10
41 10
42 10
43 10
44 10
45 10
46 10
47 10
48 10
49 10
50 10
51 10
52 10
53 10
54 10
55 10
56 10
57 10
58 10
59 10
60 10
61 10
62 10
63 10
64 10
65 10
66 10
67 10
68 10
69 10
70 10
71 10
72 10
73 10
74 10
75 10
76 10
77 10
78 10
79 10
80 10
81 10
82 10
83 10
84 10
85 10
86 10
87 10
88 10
89 10
90 10
91 10
92 10
93 10
94 10
95 10
96 10
97 10
98 10
99 10

```

73

Estación meteorológica

- Decide si hubo al menos n días consecutivos con temperaturas mayores a t .

2.5	11.1	8.2	4.0	0.1	-0.2	4.8	5.2	5.1	-0.2
-----	------	-----	-----	-----	------	-----	-----	-----	------

si $t = 4.5$ y $n = 3$

cont $\leftarrow 2$

```

00 00
01 00
02 01
03 01
04 10
05 10
06 10
07 10
08 10
09 10
10 10
11 10
12 10
13 10
14 10
15 10
16 10
17 10
18 10
19 10
20 10
21 10
22 10
23 10
24 10
25 10
26 10
27 10
28 10
29 10
30 10
31 10
32 10
33 10
34 10
35 10
36 10
37 10
38 10
39 10
40 10
41 10
42 10
43 10
44 10
45 10
46 10
47 10
48 10
49 10
50 10
51 10
52 10
53 10
54 10
55 10
56 10
57 10
58 10
59 10
60 10
61 10
62 10
63 10
64 10
65 10
66 10
67 10
68 10
69 10
70 10
71 10
72 10
73 10
74 10
75 10
76 10
77 10
78 10
79 10
80 10
81 10
82 10
83 10
84 10
85 10
86 10
87 10
88 10
89 10
90 10
91 10
92 10
93 10
94 10
95 10
96 10
97 10
98 10
99 10

```

74

Estación meteorológica

- Decide si hubo al menos n días consecutivos con temperaturas mayores a t .

2.5	11.1	8.2	4.0	0.1	-0.2	4.8	5.2	5.1	-0.2
-----	------	-----	-----	-----	------	-----	-----	-----	------

si $t = 4.5$ y $n = 3$

cont $\leftarrow 3$

Se satisface la propiedad

```

00 00
01 00
02 01
03 01
04 10
05 10
06 10
07 10
08 10
09 10
10 10
11 10
12 10
13 10
14 10
15 10
16 10
17 10
18 10
19 10
20 10
21 10
22 10
23 10
24 10
25 10
26 10
27 10
28 10
29 10
30 10
31 10
32 10
33 10
34 10
35 10
36 10
37 10
38 10
39 10
40 10
41 10
42 10
43 10
44 10
45 10
46 10
47 10
48 10
49 10
50 10
51 10
52 10
53 10
54 10
55 10
56 10
57 10
58 10
59 10
60 10
61 10
62 10
63 10
64 10
65 10
66 10
67 10
68 10
69 10
70 10
71 10
72 10
73 10
74 10
75 10
76 10
77 10
78 10
79 10
80 10
81 10
82 10
83 10
84 10
85 10
86 10
87 10
88 10
89 10
90 10
91 10
92 10
93 10
94 10
95 10
96 10
97 10
98 10
99 10

```

75

Estación meteorológica

- Decide si hubo al menos n días consecutivos con temperaturas mayores a t .

2.5	11.1	8.2	4.0	0.1	-0.2	4.8	5.2	5.1	-0.2
-----	------	-----	-----	-----	------	-----	-----	-----	------

si $t = 4.5$ y $n = 4$

cont $\leftarrow 3$

No se satisface la propiedad

```

00 00
01 00
02 01
03 01
04 10
05 10
06 10
07 10
08 10
09 10
10 10
11 10
12 10
13 10
14 10
15 10
16 10
17 10
18 10
19 10
20 10
21 10
22 10
23 10
24 10
25 10
26 10
27 10
28 10
29 10
30 10
31 10
32 10
33 10
34 10
35 10
36 10
37 10
38 10
39 10
40 10
41 10
42 10
43 10
44 10
45 10
46 10
47 10
48 10
49 10
50 10
51 10
52 10
53 10
54 10
55 10
56 10
57 10
58 10
59 10
60 10
61 10
62 10
63 10
64 10
65 10
66 10
67 10
68 10
69 10
70 10
71 10
72 10
73 10
74 10
75 10
76 10
77 10
78 10
79 10
80 10
81 10
82 10
83 10
84 10
85 10
86 10
87 10
88 10
89 10
90 10
91 10
92 10
93 10
94 10
95 10
96 10
97 10
98 10
99 10

```

76

Estación meteorológica

```

Algoritmo huboNConMayores
DE t, n

para cada día del periodo y
    mientras no se verifique la propiedad
        si la temperatura es mayor a t
            incrementa el contador
        sino
            empieza a contar nuevamente

```

Hemos establecido una condición muy general para la iteración.

```

00 00
01 00
02 01
03 01
04 10
05 10
06 10
07 10
08 10
09 10
10 10
11 10
12 10
13 10
14 10
15 10
16 10
17 10
18 10
19 10
20 10
21 10
22 10
23 10
24 10
25 10
26 10
27 10
28 10
29 10
30 10
31 10
32 10
33 10
34 10
35 10
36 10
37 10
38 10
39 10
40 10
41 10
42 10
43 10
44 10
45 10
46 10
47 10
48 10
49 10
50 10
51 10
52 10
53 10
54 10
55 10
56 10
57 10
58 10
59 10
60 10
61 10
62 10
63 10
64 10
65 10
66 10
67 10
68 10
69 10
70 10
71 10
72 10
73 10
74 10
75 10
76 10
77 10
78 10
79 10
80 10
81 10
82 10
83 10
84 10
85 10
86 10
87 10
88 10
89 10
90 10
91 10
92 10
93 10
94 10
95 10
96 10
97 10
98 10
99 10

```

77

Estación meteorológica

```

Algoritmo huboNConMayores
DE t, n

para cada día del periodo y
    mientras no se encuentren n temperaturas
        consecutivas mayores a t
        si la temperatura es mayor a t
            incrementa el contador
        sino
            empieza a contar nuevamente

```

```

00 00
01 00
02 01
03 01
04 10
05 10
06 10
07 10
08 10
09 10
10 10
11 10
12 10
13 10
14 10
15 10
16 10
17 10
18 10
19 10
20 10
21 10
22 10
23 10
24 10
25 10
26 10
27 10
28 10
29 10
30 10
31 10
32 10
33 10
34 10
35 10
36 10
37 10
38 10
39 10
40 10
41 10
42 10
43 10
44 10
45 10
46 10
47 10
48 10
49 10
50 10
51 10
52 10
53 10
54 10
55 10
56 10
57 10
58 10
59 10
60 10
61 10
62 10
63 10
64 10
65 10
66 10
67 10
68 10
69 10
70 10
71 10
72 10
73 10
74 10
75 10
76 10
77 10
78 10
79 10
80 10
81 10
82 10
83 10
84 10
85 10
86 10
87 10
88 10
89 10
90 10
91 10
92 10
93 10
94 10
95 10
96 10
97 10
98 10
99 10

```

78

Estación meteorológica

```

Algoritmo huboNConMayores
DE t, n

para cada dia del periodo y
    mientras el contador es menor a n
        si la temperatura es mayor a t
            incrementa el contador
        sino
            empieza a contar nuevamente
    Computa true si el contador es igual a n
    
```

Esta versión es más refinada y más específica que las anteriores.

79

Estación meteorológica

```

public boolean huboNConMayores(float t,int n){
    /*Decide si hubo al menos n temperaturas consecutivas mayores
    a t*/
    int cont = 0;
    for (int i=0; i < cantDias() && cont < n; i++)
        if (tMin[i] > t)
            cont++;
        else
            cont = 0;
    return cont == n;
}
    
```

Recorrido NO Exhaustivo

Establecer Casos de Prueba considerando especialmente que las n temperaturas mayores a t estén en las primeras posiciones o en las últimas de la estructura.

80

Estación meteorológica

```

public static TempMinEstacion leerTempMinEst(int cant){
    float t ;
    Scanner sca= new Scanner (System.in);
    TempMinEstacion e = new TempMinEstacion(cant);
    for (int i = 1; i <= e.cantDias();i++){
        System.out.println("Ingrese la " +
            "temperatura del día " + i);

        t = sca.nextFloat();
        e.establecerTempMin(i, t);
    }
    return e;
}
    
```

81

Estación meteorológica

```

public static void mostrarTempMinEst(TempMinEstacion e){
    for (int i = 1; i <= e.cantDias(); i++)
        System.out.println(" " + e.obtenerTempMin(i));
}
    
```

82